When Does Quantization *Save* Energy?

> An empirical practitioner's guide to the crossover effect across GPU generations.

Practitioner Edition · v1.0 Hongping Zhang · 2026-05-09 Companion to: EcoCompute Skill `v3.0.7` · Dataset `zenodo.org/records/18900289` (2025-Q1)

> **Status notice — please read.** This document is a **working paper companion**. The peer-reviewed academic version of the underlying study is in submission. **All Pro Bundle buyers automatically receive the published version free of charge** — no action needed when it goes live.

The 30-second answer

Quantization does **not** universally save energy. Direct NVML power measurement (100 ms sampling) across 360+ inference configurations on **RTX 4090D (Ada) and A800 (Ampere)** with partial **RTX 5090 (Blackwell)** coverage reveals a **per-architecture crossover threshold** — a parameter count below which quantization actually *increases* energy.

One line of config. 15 – 60 % less energy. MEASURED.

Fixing the INT8 default-threshold misuse alone (`llm_int8_threshold=0.0`) recovers **15 – 60 %** of the energy a wrong default wastes — derived directly from the **+17 % to +147 %** overhead measured in Part 3.

REGION	BEHAVIOUR	MAGNITUDE
Below crossover	Quantization **adds** energy	+25 ~ +55 %
Above crossover	Quantization **saves** energy	−15 ~ −23 %

Why this matters in practice: a single-percent change at request scale becomes **kWh / kg-CO₂ scale** at fleet scale (see Part 6 §6.1 for a fully-anchored example). The decision is free — you just need to know your threshold.

This guide turns "should I quantize?" from a research project into a 60-second lookup followed by a one-line config change.

§1 Crossover Threshold Table (architecture-specific)

Source: SKILL.md v3.0.7 · zenodo.org/records/18900289 · 2025-Q1.

GPU	ARCHITECTURE	NF4 CROSSOVER	INT8 CROSSOVER	CONFIDENCE
Tesla T4	Turing	~3.2 B	~4.0 B	★★☆ same-arch extrapolation
RTX 4090D	Ada Lovelace	~3.9 B	~4.6 B	★★★ direct measured
A800	Ampere (server)	~3.7 B	~4.3 B	★★★ direct measured
RTX 5090	Blackwell	~5.2 B	~5.6 B	★★☆ partial — full series pending public release

Confidence legend: ★★★ direct measured · ★★☆ same-architecture extrapolation · ★☆☆ cross-architecture inference. Confidence drops one step automatically if your stack is materially newer than the measurement window (PyTorch 2.4–2.12, bitsandbytes 0.45, transformers 4.47+, CUDA 12.1–12.8).

Not in the measured set (A100, H100, V100, RTX 3090, MI300X, Apple Silicon, etc.): the EcoCompute Skill maps your hardware to the nearest measured architecture and tags every numeric value as ★☆☆ **cross-architecture inference** with a ±15–25 % range band. Do not transfer numbers blindly.

§2 60-Second Decision Path

```
Q1.  What is your model's parameter count?
     └── Hugging Face model card → "n_params" field
       OR  count via transformer layers × hidden dim².

Q2.  Which GPU are you deploying on?
     └── Match its architecture row in §1.
       Off-table GPU? Use closest architecture, drop one confidence step.

Q3.  Is your model size above the crossover threshold for your
     {GPU × precision} pair?

          ┌── YES → Quantize. Expect -15 to -23 % energy.
          │       NF4 preferred over INT8 from energy perspective.
          │       Skip to Part 4 for the one-line snippet in your framework.
          │
          └── NO  → Keep FP16. Quantization will cost +25 to +55 % more.
                Direct your audit at dynamic batching and outlier
                thresholds instead — see Part 3.
```

Default assumptions when an input is missing: **FP16 baseline · batch size 1** (conservative single-request). Both flagged explicitly when surfacing recommendations.

PART 3 — ANTI-PATTERN AUDIT (SELF-CHECK IN 5 MINUTES)

Source: SKILL.md v3.0.7 § "Anti-Pattern Library — Measured" + supplementary list. Severity ranked by measured overhead. Apply top to bottom.

🔴 [HIGH · ★★★ measured] FP8 in eager mode — **+158 % to +701 %** overhead

Running FP8 quantized weights through a non-fused execution path (e.g. `torchao` without `torch.compile`) triggers per-kernel dequantization at every operator. Launch overhead dominates the math. **Fix**: serve via **vLLM** or **SGLang** — both fuse FP8 dequant into the attention / FFN kernels.

🔴 [HIGH · ★★★ measured] INT8 default outlier threshold — **+17 % to +147 %** overhead

`bitsandbytes` defaults to `llm_int8_threshold=6.0`, which routes outlier columns through a costly mixed-precision path. For models where outliers are rare, this is pure waste. **Fix**: `BitsAndBytesConfig(load_in_8bit=True, llm_int8_threshold=0.0)`.

🟡 [MED · ★★★ measured] NF4 on a sub-crossover model — **+11 % to +29 %** overhead

Quantizing below the architecture-specific NF4 crossover (Part 2 §1) costs more than it saves. The dequantization fixed cost dominates at small scales. **Fix**: keep FP16 (or BF16) for sub-crossover models.

🟡 [MED · ★★★ measured] Batch size = 1 single-request inference — **+95.7 %** per-request

A single in-flight request cannot amortize per-step fixed costs. Energy per request nearly doubles vs batch-8. **Fix**: enable dynamic batching on your serving framework; target batch 8–16.

Supplementary patterns (engineering convention, **not measured by EcoCompute**)

These are surfaced for completeness. The skill labels them `Source: engineering convention, not measured by EcoCompute` and does not assign a measured overhead figure. Treat as audit hints, not benchmarks.

- **FP32 KV cache on a quantized model** → likely bandwidth waste; prefer FP8 KV cache where supported.
- `attn_implementation="eager"` → likely missed optimization; switch to **SDPA** or **FlashAttention-2**.
- **Reloading the model per request** → init overhead; use a **persistent worker** pattern.

> Why severity-split matters: confidence-honesty is the unique selling point of this report. We never inflate engineering folklore into "measured" findings.

PART 4 — CONFIGURATION RECIPES (5 FRAMEWORKS, SAME INTENT)

Source: SKILL.md v3.0.7 § "Framework Integration Mappings". For each recommendation, the **same intent** is translated into your serving stack.

Recipe A — NF4 4-bit (or nearest 4-bit equivalent on GGUF stacks)

Bit-identical NF4 on `transformers+bnb` / `vLLM` / `TGI`. GGUF's `Q4_K_M` (Ollama, llama.cpp) is the closest 4-bit analog — absolute J/req figures here are within ±3 % only on bnb-based stacks; GGUF analogs reproduce the **direction** of savings but not the exact magnitude.

```
# transformers + bitsandbytes
from transformers import BitsAndBytesConfig
import torch

cfg = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_use_double_quant=True,
    bnb_4bit_compute_dtype=torch.float16,
)
```

```
# vLLM
vllm serve <model-id> \
    --quantization bitsandbytes \
    --load-format bitsandbytes \
    --dtype half

# Text Generation Inference (TGI)
text-generation-launcher \
    --model-id <model-id> \
    --quantize bitsandbytes-nf4

# Ollama (Modelfile, GGUF analog — not bit-identical to NF4)
PARAMETER quantization q4_K_M

# llama.cpp (GGUF analog)
llama-cli -m <model.gguf> -q Q4_K_M
```

Recipe B — INT8 with tuned threshold

The single most impactful one-line fix in this report.

```
# transformers + bitsandbytes — the canonical fix
BitsAndBytesConfig(load_in_8bit=True, llm_int8_threshold=0.0)   # avoids +17 % – +147 % default-threshold overhead
```

vLLM / TGI do not expose `llm_int8_threshold`**.** Two production workarounds:

```
# (1) Pre-quantize once with bnb, then serve the saved checkpoint:
python -c "from transformers import AutoModelForCausalLM as M, BitsAndBytesConfig as B; M.from_pretrained('<m>',
quantization_config=B(load_in_8bit=True, llm_int8_threshold=0.0)).save_pretrained('<out>')"
vllm serve <out> --quantization bitsandbytes --dtype half

# (2) Or switch to a scheme vLLM/TGI natively serve (avoids the bnb threshold question entirely):
#     GPTQ → --quantization gptq        AWQ → --quantization awq     (re-validate on your model)

# llama.cpp INT8 analog
llama-cli -m <model.gguf> -q Q8_0
```

Recipe C — FP8 (Blackwell / Hopper only)

```
# vLLM
vllm serve <model-id> --quantization fp8 --kv-cache-dtype fp8

# TGI
text-generation-launcher --model-id <model-id> --quantize fp8

# TensorRT-LLM — enable in build config
# build_config.fp8_qat = True
```

> The Pro Bundle is a **living artifact**, not a snapshot. This page is your contract: what you get today, what is being measured next, and the free-upgrade promise.

§5.1 What's in this Bundle (v1.0, today)

ASSET	DESCRIPTION
This PDF (`Practitioner_Report_v1.pdf`)	The 6-part practitioner guide you are reading
`Reproducible.ipynb`	One-click Jupyter notebook reproducing every measured figure on your own machine
`configs/{bnb,vllm,tgi,ollama,llamacpp,trt}.txt`	5 framework one-line snippets, ready to copy
`Anti_Pattern_Checklist.pdf`	A4 single-page audit checklist (printable)
`fleet_calc.ipynb`	Plug your traffic numbers, get your annual kWh / CO_2 delta
`README.md`	License terms, 30-day re-download policy, errata channel

The 360-row open dataset (CC-BY 4.0) stays free on Zenodo; it is *not* repackaged in this Bundle. The Pro Bundle's value is the **derivative analysis layer** above it.

§5.2 What's measured next (Bundle v1.1, target ~Q3 2026)

These are coming. **You do not need to repurchase**; see §5.3.

DIRECTION	STATUS	WHAT IT ADDS
Full RTX 5090 series (FP16 / NF4 / INT8 / FP8 across 1B – 13B models)	in progress	Promotes RTX 5090 rows from ★★☆ partial → ★★★ direct measured
FP8 expansion across 5 model families (Qwen, Llama, Mistral, Phi, Gemma)	queued	Closes the FP8 gap on Ada / Ampere
Multi-GPU Tensor-Parallel (TP=2, TP=4) on A800 / RTX 4090D	queued	Enables 70B-class advisory grounded in real measurement
Long-context (8 k / 16 k / 32 k prompt) energy profiles	queued	Unlocks RAG / long-document deployment guidance

§5.3 Free-Upgrade Promise

Every Bundle v1.0 buyer's email is on the upgrade list. When v1.1, v1.2, … and the **peer-reviewed paper PDF** ship, you receive a download link via the same channel as your original purchase — **no extra payment, no action required**.

This is the central promise of the Pro Bundle: you are buying a ticket to the **maturing dataset**, not a static file.

§5.4 Methodology brief

All measured rows are collected with **NVML power sampling at 100 ms resolution**, GPU isolated to one process, request shape standardized at **batch 1, prompt 512, max-new-tokens 128, FP16 baseline**. Each {GPU × Model × Precision} cell is **averaged over multiple runs after warm-up**, with statistical outliers excluded. Stack window: PyTorch 2.4–2.12, bitsandbytes 0.45, transformers 4.47+, CUDA 12.1–12.8 (Q1 2025). Per-cell run counts, warm-up procedure, and outlier-rejection criteria are documented in `Reproducible.ipynb` along with raw CSVs and reproduction commands.

> **Stack-version drift caveat**: if `bitsandbytes ≥ 0.48` or `transformers ≥ 4.55` , figures are downgraded one confidence step. Re-run **Reproducible Notebook** to verify.

§6.1 Fleet impact — a fully-anchored reference scenario

Anchor: a production deployment running **A800 × Qwen2-7B**, **1 000 000 requests/day** (typical mid-size SaaS), US grid carbon intensity **390 g CO₂/kWh**, electricity price **$0.12/kWh**.

Measured baselines (SKILL.md v3.0.7 lookup table, J/request):

- FP16: **89.4 J/req** · NF4: **58.7 J/req** · INT8 (threshold=0): **63.2 J/req** · FP8: **67.8 J/req**

At the anchor traffic level, switching FP16 → NF4 saves **30.7 J/request**. Multiplied out:

SWITCH	Δ /YEAR	Δ COST/YEAR	Δ CO_2/YEAR
FP16 → NF4	**≈ −3 110 kWh**	**≈ −$373/yr**	**≈ −1.2 t**
FP16 → INT8 (threshold=0)	≈ −2 656 kWh	≈ −$319/yr	≈ −1.0 t
FP16 → FP8 (Hopper/Blackwell)	≈ −2 190 kWh	≈ −$263/yr	≈ −0.85 t

For a single Qwen2-7B-class model at 1 M req/day, a one-line precision change is worth roughly $370/year and ~1.2 tonnes CO_2/year per model. The arithmetic is linear: at **100 K req/day** divide by 10 (≈ $37/yr, ≈ 122 kg/yr); at **10 M req/day** multiply by 10 (≈ $3 700/yr, ≈ 12 t/yr); a fleet of 10 deployed models at the anchor scale stacks to **~$3 700/yr and ~12 t CO_2/yr**.

> **Plug your real numbers** — traffic, request length, model, grid intensity, electricity price — into `fleet_calc.ipynb` (in the Pro Bundle) to get your exact figure. The anchor above is illustrative; SaaS deployments serving long-form generation or RAG see materially larger absolute savings.

Two more anti-pattern audit-fix opportunities (per Part 3) typically stack on top of the precision decision:

- **Fix INT8 default-threshold misuse** (if applicable): 15 % – 60 % per request recovered — same arithmetic structure as above.
- **Enable dynamic batching (BS=1 → BS=8)**: derived from the measured +95.7 % BS=1 overhead, eliminating it cuts per-request energy roughly in half on top of any precision gains.

§6.2 Boundary conditions — re-validate when:

- **Model > 14 B params** → ±20 % extrapolation; treat as ★★☆.
- **Non-NVIDIA hardware** → no measured data; do not transfer.
- **Multi-GPU (TP / PP)** → single-GPU benchmarks only; v1.1.
- **Custom fine-tuned weights** → activation distributions may differ.
- **Stack newer than window** (`bitsandbytes ≥ 0.48` / `transformers ≥ 4.55`) → ★★★ → ★★☆ downgrade.

§6.3 Provenance & citation

- **Dataset (CC-BY 4.0)**: zenodo.org/records/18900289 · **Code**: github.com/ecocompute-ai/quantization-energy-crossover
- **Working paper**: *When Does Quantization Save Energy?* — peer review pending.
- **Cite as**: Zhang, H. (2026). *EcoCompute Practitioner Report v1.0*. Dataset: zenodo.org/records/18900289.

§6.4 License & support

- Licensed for **immediate team use** (≤ 10 persons) of `demo@ecocompute.dev` . Team-wide rollout requires Team License — contact zhanghongping1982@gmail.com.
- **Public re-hosting prohibited**. Open dataset (Zenodo) remains CC-BY 4.0 and free.
- **Inquiries**: zhanghongping1982@gmail.com.

YOUR NEXT 60 SECONDS

1. Open **Part 2 §1** → find your GPU row.
2. Check model size vs NF4 / INT8 crossover threshold.
3. *Above* threshold → copy **Recipe A** from Part 4. *Below* → keep FP16, audit Part 3.
4. Re-validate with `Reproducible.ipynb`. Done.

EcoCompute Practitioner Report · ECC-DEMO-0001 · Licensed to demo@ecocompute.dev · 2026-05-09 · Dataset zenodo.org/records/18900289 · © 2026 Hongping Zhang